SYNAP Architecture

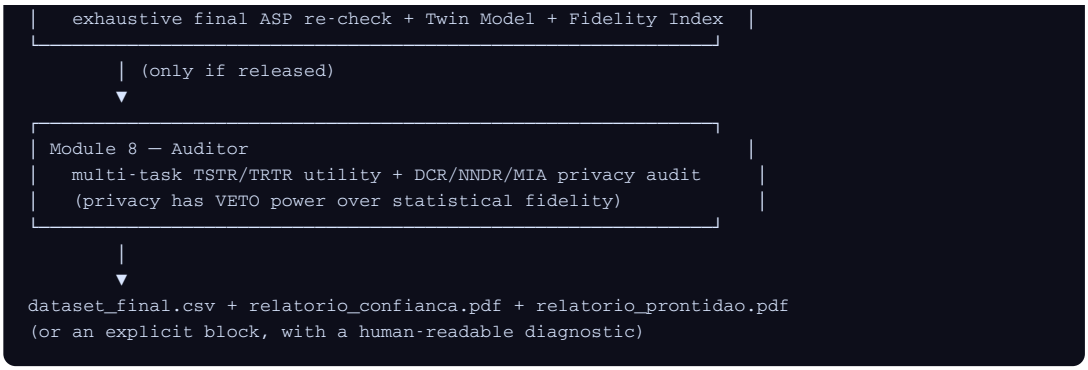
Architecture

This document describes how S.Y.N.A.P. is built: the design philosophy, each of the 8 modules, the orchestrator, the web layer, and the testing strategy. For *why* the project exists and how it compares to other synthetic-data tools, see `PROJECT_NARRATIVE.md`. For empirical proof that the system does what this document claims, see `validation/Relatorio_Validacao_Fiabilidade_SYNAP.pdf`.

---

1. Overview

S.Y.N.A.P. is not a single generative model wrapped in a CLI. It is eight specialized modules, each with one responsibility and an explicit public interface, composed into a loop that generates, validates, and corrects data continuously — rather than a linear pipeline that generates first and checks later.



Each module lives in its own Python package under `src/synap/`, with its own `models.py` (Pydantic data structures), `exceptions.py` (module-specific error types), and `tests/` (in the top-level `tests/` directory, mirroring the package name). A module's public interface is exactly what its `__init__.py` exports — nothing else is meant to be imported by other modules or by users.

---

2. Design Philosophy

A few decisions recur across all 8 modules and are worth stating once, rather than repeating in every section:

- **Never fail silently.** Every module defines its own exception hierarchy (`ModuleError` → `ModuleValidationError`, etc.) and raises explicit, actionable errors instead of returning `None` or swallowing exceptions.
- **Never claim what wasn't tested.** If privacy can't be evaluated (no real holdout provided), the system says so explicitly rather than returning a reassuring default. This is not a style preference — it's enforced behavior, covered by tests (see Module 8).
- **State the assumption, don't hide the limitation.** Where a simplification was made for scope reasons (e.g., the simplified PC-algorithm in Module 2, or the univariate-only ILP template in Module 4), it is documented in the module's docstring and in Section 14, not silently shipped as if it were complete.
- **Composition over reimplementation.** Module 5 does not duplicate Module 3's filtering logic — it imports and reuses Module 3's public functions directly. Module 8 reuses Module 2's `partial_correlation` and `fisher_z_p_value`. When two modules need the same statistical primitive, one imports it from the other; it is never copy-pasted.
- **Immutability where it matters.** `StatisticalContract` and `CausalDAG` are frozen Pydantic models — once a batch of rows is being generated against a Contract, that Contract cannot change underneath it. The few genuinely mutable objects in the system (`HipotalamoState`, `HomeostaseState`, `RunningCovarianceAccumulator`) are mutable by necessity (incremental state across checkpoints) and are documented as the deliberate exceptions.

---

3. Module 1 — Neocortex (`synap.neocortex`)

Role: the entry point. Turns a user-provided schema (YAML/JSON) plus either real anchor data (CSV) or manual statistical parameters into a **Statistical Contract** — an immutable object that is the single source of truth for every module downstream.

Key behavior: - Parses column definitions (numeric with `min`/`max`, or categorical with a fixed

category list) and business rules (`"column operator value"` strings, e.g. `"income > 0"`). - With a real CSV anchor (≥ 50 rows required, ≥ 100 recommended): computes per-column mean/std or category frequencies, and the full Pearson correlation matrix between numeric columns. - Without any anchor: falls back to **Synthetic Pure Mode** — standard normal/uniform distributions — and attaches an explicit warning to the Contract. This is never silent. - Cross-validates manual parameters against declared domains (e.g., rejects a manually-specified mean outside a column's declared `[min, max]`).

Public interface: `build_contract(...) -> StatisticalContract` , `load_schema` , `parse_business_rules` .

---

4. Module 2 — Hippocampus (`synap.hipocampo`)

Role: causal structure and the initial seed. Validates a user-supplied causal DAG (or learns one) against the Contract's correlation matrix, then generates 5,000–10,000 rows that respect both the marginal distributions and the causal structure.

Key behavior: - **D-separation testing:** for a user-provided DAG, tests whether non-adjacent node pairs are truly conditionally independent given their parents (partial correlation + Fisher-Z test). If a real dependency is found that the DAG doesn't encode, the missing edge is added automatically, with a reported BIC improvement. - **DAG learning:** without a user-provided DAG, runs a simplified PC-algorithm (skeleton phase + v-structure orientation + a declared-order fallback for the remaining edges) — explicitly *not* a full implementation of Meek's rules (see Section 14). - **Seed generation:** topological sampling — root nodes drawn from their marginal distribution, child nodes drawn via a linear-Gaussian model whose coefficients are solved algebraically from the target correlation matrix (multiple regression in closed form, not gradient descent). - Categorical columns with a numeric parent in the DAG are sampled via percentile mapping against that parent's distribution — this is how the system preserves category-numeric dependence (documented as requiring an explicit DAG edge; see Section 14).

Public interface: `run_hipocampo(contract, user_dag_edges=None, causal_order=None, n_rows=5000) -> HipocampoOutput` (contains `dag` , `seed` , `warnings` , `seed_report`).

---

5. Module 3 — Expander (`synap.expansor`)

Role: the generative engine. Grows an in-memory pool of approved rows by recombining columns from `k` (3–5) randomly-selected source rows into new "chimera" rows, filtered before they're ever shown downstream.

Key behavior: - **Mahalanobis distance filter:** rejects any candidate row whose numeric values are statistically implausible relative to the Contract's target mean vector and covariance matrix (default threshold: 3 standard deviations). - **Plausibility scorer:** a Random Forest trained to distinguish real seed rows (label 1) from the same rows with each column independently shuffled (label 0) — a density-ratio classifier trick that estimates "does this specific combination of values look like it belongs together", not just "are the marginals right". - **Negative attention:** rejected rows immediately reduce the selection weight of the specific source rows that produced them, so the same mistake becomes less likely for the rest of the batch — real-time adaptation, not a static filter. - **Reservoir-capped memory:** the growing pool of approved rows is capped (`MAX_MEMORY_SIZE` , default 50,000) via reservoir sampling, so memory usage stays bounded even when generating millions of rows.

Public interface: `run_expansor(seed, contract, n_rows, ...) -> ExpansorOutput`. Also exposes its filtering primitives individually (`mahalanobis_distance` , `train_plausibility_model` , `generate_chimera_row` , `penalize_sources`) — Module 5 imports these directly rather than reimplementing them.

---

6. Module 4 — Hypothalamus (`synap.hipotalamo`)

Role: logical and causal validation. Has two genuinely distinct responsibilities that are easy to conflate but serve different purposes:

(a) Strict rule validation (ASP-style): vectorized checking of every business rule against every row — no shortcuts, no sampling. This part is composed directly into Module 5's per-row loop (`validate_row`).

(b) Continuous causal monitoring + rule induction (ILP): this part runs separately, once per checkpoint (not per row): - **Drift detection:** recomputes the correlation matrix over accumulated approved rows and re-runs the same d-separation tests from Module 2 — if a relationship the DAG assumed doesn't exist has emerged, or one that should hold has weakened, it's flagged. - **Rule induction:** searches for candidate rules of the form `IF numeric_column > threshold THEN categorical_column = value` , restricted to a univariate template (see Section 14), requiring both statistical support (≥ 100 examples) and — critically — zero logical exceptions when the candidate is tested as a hard constraint. A rule that duplicates an existing DAG edge is discarded, not proposed (this is a tested guarantee, not a heuristic). - Uses `RunningCovarianceAccumulator` : sufficient statistics (sums and cross-products) updated incrementally, so correlation drift can be monitored over millions of rows without storing them all.

Public interface: `validate_row` , `validate_batch` (part a); `run_hipotalamo(new_batch, contract, dag, state, checkpoint_size, ...) -> HipotalamoOutput` (part b), where `HipotalamoState` is the module's one deliberately mutable object.

---

7. Module 5 — Cognitive Core (`synap.nucleo`)

Role: the "dual-pathway loop" that gives the system its name — the point where Modules 3 and 4's real-time checks and a novelty detector are composed into a single per-row cycle.

Key behavior: - **Autoencoder novelty detection:** a small MLP (bottleneck architecture) trained to reconstruct rows from the seed. A candidate whose reconstruction loss exceeds `mean + 3·std` of the seed's own loss distribution is flagged as an artifact — even if it already passed the Mahalanobis and plausibility filters (this catches cross-type inconsistencies, e.g. a numeric/categorical combination that's globally implausible even though each value is individually fine). - **Composed, not reimplemented:** `run_nucleo` imports Module 3's `generate_chimera_row` , `mahalanobis_distance` , `train_plausibility_model` , and `penalize_sources` directly, and Module 4's `validate_row` directly. This was an explicit integration decision, made and confirmed before building the module, to avoid duplicating Module 3's already-tested filtering logic. - **Differentiated penalties:** an artifact (autoencoder) penalizes its source rows more aggressively than a logical rejection (ASP), reflecting higher confidence that the *combination of sources*, not chance, is the problem. - **Cycle chaining:** `NucleoOutput` returns both the updated memory and the updated source-selection weights, so successive batches can chain learned negative attention across calls rather than resetting it.

Public interface: `run_nucleo(seed, contract, n_rows, dag=None, initial_weights=None, ...) ->`

`NucleoOutput`, `train_autoencoder`.

---

8. Module 6 — Homeostasis Controller (`synap.homeostase`)

Role: prevents the generator from drifting away from the Contract or collapsing into a low-diversity niche — the two failure modes that a purely local, per-row filtering approach (Modules 3–5) cannot catch on its own, because they only become visible in aggregate.

Key behavior: - **EMA statistics, not cumulative.** An earlier version of this module used all-time cumulative statistics (reused from Module 4's `RunningCovarianceAccumulator`) and it was mathematically unable to correct a drift within a bounded number of new rows, because a large historical base dilutes any correction. The shipped version uses an exponential moving average of each checkpoint's own batch statistics — intentionally recency-weighted, because a homeostasis system has to react to the *current* state of the generator, not its entire history. - **KL-divergence** (closed-form Gaussian approximation) per numeric column, transformed to a bounded `[0, 1)` score via `1 - exp(-KL)` so the 5%/10% thresholds from the spec read naturally as percentages. - **Shannon entropy** (normalized) per categorical column, to catch diversity collapse independently of mean/variance drift. - **Reweighting by likelihood ratio**, not by naive z-score. An earlier version boosted "values above the current mean" without bound, which corrected the mean but inflated the variance uncontrollably. The shipped version reweights source rows by the ratio of target-Gaussian density to current-Gaussian density (an importance-sampling correction), which corrects mean and variance together. - **Multiplicative composition, not replacement.** The weights this module returns are meant to be multiplied into Module 3/5's existing selection weights across checkpoints (`weights *= homeostasis.pesos_multiplicador`), not to replace them — the correction accumulates.

Public interface: `run_controlador_homeostasia(new_batch, contract, memory, state, ...) -> HomeostasisOutput`.

---

9. Module 7 — Compiler (`synap.compilador`)

Role: the first gatekeeper. Re-validates the entire final dataset from scratch (no shortcuts inherited from the generation loop), trains a lightweight "Twin Model" to prove predictive equivalence with real data, and either releases the CSV or blocks it with an explicit diagnostic.

Key behavior: - **Exhaustive final ASP check** on 100% of rows — a genuine safety net, independent of Module 4's per-batch checks. - **Twin Model:** Logistic Regression (classification) or Linear Regression (regression) — deliberately simple and deterministic, not XGBoost or a deep model, because the model's job is to attest to basic statistical realism, not to maximize predictive performance. - **Fidelity Index:** trains the Twin Model on the synthetic data, tests it on a real holdout (Train-Synthetic-Test-Real), and compares against the same model trained and tested purely on real data (Train-Real-Test-Real). The Index is `1 - relative_difference`, generalized identically for AUC (higher-is-better) and RMSE (lower-is-better). - **Order matters:** ASP validation runs before the Twin Model — an already-invalid dataset never wastes time training a model. - Without a real holdout, the Fidelity Index cannot be computed at all, and the output is blocked by default (overridable with an explicit `forcar_saida` flag, which is always reported in the output warnings).

Public interface: `run_compilador(dataset, contract, target_column=None, real_holdout=None, fidelity_threshold=0.90, ...) -> CompilerOutput`.

10. Module 8 — Auditor (`synap.auditor`)

Role: the second gatekeeper, and the one most existing synthetic-data tools skip. Proves two things Module 7 alone cannot: multi-task utility, and privacy — with privacy holding **veto power** over everything else, including a passing Fidelity Index.

Key behavior: - **Multi-task TSTR/TRTR.** Module 7's Fidelity Index uses exactly one target column. Module 8 evaluates several target columns of different types (classification and regression) — the validation report shows a real case where a single-task Fidelity Index of 0.9999 coexisted with a multi-task utility retention of only 75.2%, because a regression task behaved much worse than the classification task the single-task check happened to use. - **Distance to Closest Record (DCR):** median distance from each synthetic row to its nearest real neighbor, compared against an internal real-vs-real baseline (splitting the holdout in half). A ratio well below the baseline suggests memorization. - **Nearest-Neighbour Distance Ratio (NNDR):** a second, independent signal against the same failure mode, less prone to the false positives a lone DCR check can produce. - **Exact-match check:** literal row-level duplication between synthetic output and real holdout, at high numeric precision (a coarse rounding tolerance was tried first and produced false positives on independent continuous data — see Section 14). - **Membership Inference Attack (MIA):** a distance-based attacker trained to distinguish rows that *did* inform the generator (`training_reference`) from rows that never did (`real_holdout`), using each row's distance to its nearest synthetic neighbor as the only feature. An AUC near 0.5 means the attack fails — the generator isn't leaking membership information. Without `training_reference`, this specific check is explicitly marked *not evaluated*, never assumed safe by default.

Public interface: `run_auditor(dataset_sintetico, contract, real_holdout, training_reference=None, target_columns=None, ...) -> AuditorOutput`.

11. Orchestrator (`synap.main`)

Ties all 8 modules into one configurable run:

```
Module 1 → Module 2 → [ Module 5 ↔ Module 4(a) ] per batch → Module 4(b) per checkpoint
        → Module 6 per checkpoint → (repeat until target_rows) → Module 7 → Module 8
```

- `SynapConfig` (Pydantic): the single configuration object for an entire run — schema, anchor data, DAG, batch size, thresholds, output directory.
- `run_synap(config) -> SynapResult`: executes the full loop, returning per-batch summaries, the final Contract, the Compiler and Auditor outputs, and aggregated warnings prefixed by originating module (`[M4/batch 3]`, `[M6/batch 3]`) for traceability.
- **A deliberate integration decision, stated explicitly:** the DAG is never restructured mid-stream in response to a detected causal drift (Module 4b). Doing so on a DAG already in use by thousands of generated rows risks introducing cycles or instability; a persistent drift is surfaced as a strong warning, and rebuilding Module 2 is left to the user.
- Rules induced by Module 4's ILP are *not* automatically re-injected into the ASP validator used by Module 5/7 — `ConsolidatedRule` (conditional form) and `BusinessRule` (simple form) are structurally different, and bridging them was out of scope. Induced rules are surfaced in `SynapResult.warnings` for manual review.
- `serve_site(...)`: starts the local web server (see Section 12) that serves both the static repository files and the live pipeline API.

12. Web Server & Website (`synap.webserver`, `site/`)

`synap.webserver.create_app` is a Flask application, not a reimplementation of the pipeline in JavaScript:

- `GET /` and `GET /<path:filename>` serve the repository root (with a path-traversal guard restricted to the repo tree) — this is what lets `site/index.html`'s relative links to `../README.md`, `../PROJECT_NARRATIVE.md`, `../validation/*.pdf`, etc. resolve without depending on a published GitHub repository.
- `POST /api/generate` accepts a JSON schema (and optional CSV uploads via multipart/form-data), builds a real `SynapConfig`, and runs `run_synap` in a background thread — returning a `job_id` immediately rather than blocking the HTTP request for the full pipeline duration.
- `GET /api/jobs/<job_id>` is polled by the website's JavaScript to render live log lines and, once finished, the real Fidelity Index, privacy verdict, and download links.
- `GET /api/jobs/<job_id>/download/<name>` serves the actual generated `dataset_final.csv` / `relatorio_confianca.pdf` / `relatorio_prontidao.pdf` from that specific run.

`site/index.html` is a single self-contained file (no build step): a particle-network hero, an animated 8-node pipeline diagram, scroll-triggered reveals, and — functionally distinct from the decorative parts — the "Generate Data" form that calls the API above. Started via:

```
synap --serve-site
```

which serves the repository root and opens the browser automatically.

---

13. Testing & Validation Strategy

Two distinct layers, answering two distinct questions:

1. **Unit and integration tests (`pytest`, 180+ tests):** does each function behave correctly, including edge cases and error paths? Every module has its own test directory (`tests/<module>/`); `tests/test_integration.py` runs the full 8-module pipeline end-to-end with real modules, no mocking. Runs in CI on every push (`.github/workflows/ci.yml`, Python 3.11 and 3.12).
2. **The Validation & Reliability Report (`validation/`):** does the *system as a whole* recover a real statistical and causal structure it was never directly given? This uses a reference dataset with a **known ground-truth generative process** (true correlations fixed before a single row exists) — a stronger test than comparing against an arbitrary real sample, because it can distinguish "looks statistically plausible by chance" from "recovered the actual underlying structure". Formal statistical tests used: Kolmogorov-Smirnov (marginal distributions), chi-square (categorical proportions), correlation-recovery error against ground truth, TSTR/TRTR, and the full DCR/NNDR/MIA privacy battery. Findings that didn't flatter the system (3 of 4 numeric columns failing the KS test at $\alpha=0.05$; a regression task's utility retention falling below the recommended threshold) are reported, not omitted — see the report's Section 9 for the corresponding, explicit limitations.

---

14. Known Limitations

Stated here once, precisely, instead of scattered as vague caveats:

- **Causal learning (Module 2)** implements a simplified PC-algorithm — skeleton phase with a conditioning-set size capped at 2, v-structure orientation, and a declared-order fallback for remaining edges. It does **not** implement Meek's orientation rules in full.
- **Rule induction (Module 4)** is restricted to a single template (`numeric_column > threshold → categorical_column = value`), searched over a small quantile grid. It is not a general-purpose ILP engine.
- **Category-numeric dependence (Module 2) requires an explicit DAG edge.** Quantified directly in the Validation Report (Section 4): with a provided DAG, a categorical column's association with its numeric driver was recovered at 0.805 vs. 0.815 in the real data; without any DAG, the same association collapsed to 0.0004 — essentially zero.
- **Privacy claims require a real holdout, always.** No exception, no default-safe fallback. If `real_holdout` isn't provided, Module 8 reports privacy as *not evaluated*, not as passed.
- **Exact-match detection (Module 8)** compares numeric columns at high precision (6 decimal places) rather than a coarse rounding tolerance — an earlier, coarser tolerance produced false-positive "matches" on independent continuous data purely from floating-point coincidence at scale.
- **The website's "Generate Data" section requires `synap --serve-site`.** Opening `site/index.html` directly via `file://` still renders the presentation and animations, but the generation form has no server to answer `/api/generate`, and the relative links to repository documents do not resolve the same way.

---

15. Repository Layout

```
synap/
├── pyproject.toml          # packaging (src/ layout), `synap` console entry point
├── LICENSE                 # Apache 2.0
├── README.md, PROJECT_NARRATIVE.md, ARCHITECTURE.md (this file)
├── CONTRIBUTING.md, CODE_OF_CONDUCT.md, SECURITY.md, CHANGELOG.md, CITATION.cff
├── .github/                # CI workflow, issue/PR templates
├── src/synap/
│   ├── main.py              # orchestrator: SynapConfig, run_synap, CLI, serve_site
│   ├── webserver.py         # Flask API bridging the website to the real pipeline
│   ├── neocortex/           # Module 1
│   ├── hipocampo/           # Module 2
│   ├── expansor/            # Module 3
│   ├── hipotalamo/          # Module 4
│   ├── nucleo/              # Module 5
│   ├── homeostase/          # Module 6
│   ├── compilador/          # Module 7
│   └── auditor/             # Module 8
├── tests/                   # one directory per module, plus test_integration.py, webserver/
├── validation/              # Validation & Reliability Report + scripts to reproduce it
└── site/                    # website (presentation + live "Generate Data" form)
```